Technical Documentation — RAG Customer Support Assistant

Field	Value
Project	RAG-Based Customer Support Assistant with LangGraph & HITL
Author	Senior AI Engineer — Innomatics Research Labs
Date	April 2026
Version	1.0

---

1. Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture pattern that augments a Large Language Model's generation capability with an external knowledge retrieval step. Instead of relying solely on the knowledge encoded in the LLM's parameters during training — which is static, potentially outdated, and prone to hallucination — RAG injects relevant documents into the prompt at inference time, grounding the LLM's response in authoritative source material.

The architecture has three stages:

1. **Indexing** (offline): Documents are chunked, embedded into dense vectors, and stored in a vector database. This creates a searchable knowledge index that maps semantic meaning to document fragments.
2. **Retrieval** (online): When a user query arrives, it is embedded using the same model and compared against the indexed vectors via similarity search. The top-k most semantically similar chunks are returned as retrieval results.
3. **Generation** (online): The retrieved chunks are injected into the LLM prompt as context. The LLM generates a response that synthesizes information from these chunks, producing a natural language answer grounded in the source documents.

The critical insight behind RAG is the separation of knowledge storage from reasoning capability. The LLM provides language understanding and generation; the vector store provides up-to-date, verifiable knowledge. This separation makes the system updatable (re-index documents, not retrain the model), auditable (every response can be traced to source chunks), and cost-effective (no expensive fine-tuning cycles).

1.2 Why RAG Over Alternatives?

Fine-tuning encodes knowledge directly into model weights. This requires substantial compute, produces a model that is frozen at the time of training, and provides no mechanism for source attribution. When a fine-tuned model produces an incorrect answer, there is no way to determine which training example caused the error. For customer support, where policies change frequently, fine-tuning is impractical — every policy update would require a new fine-tuning run.

Prompt stuffing (concatenating entire documents into the prompt) is simple but hits hard limits: LLM context windows are finite (4K-128K tokens depending on model), and attention quality degrades with context length. For a knowledge base with hundreds of FAQ entries, prompt stuffing is not feasible — the entire corpus exceeds the context window, and even if it fit, the LLM would struggle to locate the relevant information within a massive prompt.

RAG addresses both limitations: it retrieves only the relevant subset of knowledge (typically 3-5 short chunks), keeping the prompt concise and focused; and it operates over a knowledge base that can be updated by simply re-indexing documents.

1.3 Why Customer Support?

Customer support is the canonical RAG use case because it satisfies three conditions that make RAG maximally effective:

1. **Structured knowledge base**: Support documentation is organized into discrete topics (returns, shipping, billing) with clear answers. This maps naturally to the chunk-and-retrieve paradigm.
2. **High query diversity**: Customers ask the same questions in thousands of different phrasings. Semantic similarity search handles this variation naturally — "How do I send back an item?" and "What's your return process?" map to the same chunks.
3. **Clear escalation boundaries**: Some queries (billing disputes, security incidents) inherently require human judgment. The confidence-based routing in our system maps directly to this operational reality.

---

2. System Architecture — Deep Dive

The system is composed of two decoupled pipelines: an **offline ingestion pipeline** and an **online query pipeline**. This decoupling is a deliberate design choice — it means the ingestion process (which is computationally expensive due to embedding generation) runs

independently of the query-serving path (which must be low-latency).

Component Interaction Narrative

The ingestion pipeline starts with `PyPDFLoader`, which reads PDF files page by page, producing LangChain `Document` objects. These documents flow into `RecursiveCharacterTextSplitter`, which breaks pages into 500-character chunks with 50-character overlap. The chunks then pass through the `all-MiniLM-L6-v2` embedding model, which converts each chunk's text into a 384-dimensional vector. Finally, these vectors (along with the original text and metadata) are persisted to ChromaDB's local storage.

The query pipeline is orchestrated by a LangGraph `StateGraph`. When a user submits a query, the graph initializes a `SupportState` dictionary and passes it through three nodes sequentially. The `retrieve_node` embeds the query using the same MiniLM model, searches ChromaDB for the top-3 most similar chunks, and writes the chunks and a confidence score to state. The `generate_node` constructs a grounded prompt from the retrieved context and invokes the LLM (GPT-3.5-turbo or Gemini), writing the response to state. After generation, a conditional edge inspects the confidence score and response text — if either indicates uncertainty, the graph routes to `hitl_node` for human escalation; otherwise, it terminates and returns the response.

Why Decoupling Matters

Decoupling ingestion from query serving provides three operational benefits:

1. **Independent scaling**: Ingestion can run on a beefy machine with GPU; query serving runs on a lighter instance with fast network to the LLM API.
2. **Independent deployment**: Document updates do not require restarting the query server — ChromaDB is read from disk each query, always reflecting the latest index.
3. **Failure isolation**: An ingestion failure (e.g., corrupt PDF) does not affect the query-serving path, which continues operating on the existing index.

---

3. Design Decisions

3.1 Chunk Size = 500 Characters

Decision: Set `RecursiveCharacterTextSplitter.chunk_size` to 500.

Alternatives considered:

- **250 characters**: Higher retrieval precision (smaller chunks = more focused), but chunks would often split mid-sentence, losing context.
- **1000 characters**: More context per chunk, but retrieval becomes less precise — a large chunk containing both relevant and irrelevant content scores lower on similarity, and the LLM must sift through noise.

Why 500: For FAQ-style customer support content, individual answers typically span 2-4 sentences (300-500 characters). A 500-character chunk captures a complete answer without including content from adjacent topics. Empirically, this produces chunks that align with natural paragraph boundaries in the source documents. It also fits comfortably within the embedding model's effective context (MiniLM handles up to 256 tokens ≈ ~1000 characters, so 500 characters uses ~50% of capacity, leaving room for longer sentences).

Trade-off: Some longer multi-paragraph answers (e.g., detailed return procedures) are split across 2-3 chunks, requiring the LLM to synthesize information from multiple retrieved results.

3.2 Chunk Overlap = 50 Characters

Decision: Set `chunk_overlap` to 50 (10% of chunk_size).

Alternatives considered:

- **0 overlap**: Faster chunking, fewer total chunks, but creates hard boundaries where context is lost. A sentence at a chunk boundary would be split, with neither chunk containing the complete thought.
- **100-200 overlap**: Captures more boundary context, but inflates the total chunk count significantly (~20-40% more chunks), increasing storage and retrieval latency without proportional quality improvement.

Why 50: A 10% overlap is the standard heuristic for text chunking. It captures approximately one sentence of overlap, which is enough to maintain coherence across chunk boundaries. For our FAQ content with clear paragraph breaks, most splits occur at paragraph boundaries anyway, making overlap relevant only for the minority of cases where a topic spans paragraph boundaries.

3.3 Embedding Model: all-MiniLM-L6-v2

Decision: Use `sentence-transformers/all-MiniLM-L6-v2` for all embedding operations.

Alternatives considered:

- **OpenAI text-embedding-ada-002** (1536-dim): Higher quality on some benchmarks, but costs $0.0001/1K tokens and sends all data to OpenAI's servers. For a support knowledge base that may contain sensitive customer information, this introduces both cost and privacy concerns.

- **all-mpnet-base-v2** (768-dim): Slightly higher quality on MTEB (+1.5% on average), but 3x the embedding dimension (768 vs 384) and 2x the model size (110MB vs 80MB). The quality improvement is marginal for short FAQ text.
- **BGE / E5 models**: Newer models with strong benchmarks, but require specific query prefixing ("query: " / "passage: ") and introduce a dependency on less established codebases.

Why MiniLM-L6-v2: It runs locally (zero API cost, zero data egress), produces 384-dimensional vectors (compact storage, fast similarity computation), and ranks in the top tier of MTEB for its parameter class. For short FAQ chunks (50-100 tokens), the quality difference between MiniLM and larger models is negligible.

Trade-off: For long documents or nuanced semantic queries, a larger model like `mpnet-base-v2` would produce better embeddings. The current choice optimizes for speed and simplicity at the expense of marginal quality on complex queries.

3.4 ChromaDB Over Alternatives

Decision: Use ChromaDB as the vector store with local file persistence.

Alternatives considered:

- **FAISS**: Facebook's vector similarity library. Extremely fast for pure similarity search, but provides no metadata storage, no persistence API (requires manual pickle/load), and no built-in document management. FAISS is a search index, not a database.
- **Pinecone**: Fully managed cloud vector database. Excellent for production, but requires account provisioning, introduces network latency, costs money beyond the free tier, and sends all data to a third-party cloud.
- **Weaviate**: Full-featured vector database with GraphQL API. Overkill for a single-collection use case — it requires running a separate server process and introduces infrastructure complexity.

Why ChromaDB: It provides the exact feature set needed — local persistence (data survives restarts), metadata filtering (can filter by source document), and a clean Python API — without requiring external infrastructure. Its `persist_directory` parameter makes backup and migration trivial (copy the directory). For production, ChromaDB also supports a client-server mode that separates storage from application logic.

3.5 Top-K = 3 Retrieval

Decision: Retrieve 3 chunks per query.

Alternatives considered:

- **k=1**: Minimal context — fast, but a single chunk may not contain the complete answer. If the best chunk is only partially relevant, the LLM has no fallback context.
- **k=5 or k=10**: More context, but introduces noise — lower-ranked chunks may be irrelevant, diluting the LLM's attention and increasing the chance of the LLM pulling information from an irrelevant chunk.

Why 3: Three chunks provide enough context for the LLM to synthesize a complete answer (especially when an answer spans 2 chunks due to splitting) while keeping the total context concise (~1500 characters). This fits comfortably within any LLM's context window and keeps inference fast. The average confidence score over 3 results is also more statistically stable than over 1 result.

3.6 LangGraph Over LangChain Chains

Decision: Use LangGraph `StateGraph` for workflow orchestration.

Alternatives considered:

- **LangChain SequentialChain**: Linear chain of operations. Cannot express conditional branching (HITL routing) without workarounds.
- **LCEL (LangChain Expression Language)**: Supports `RunnableBranch` for conditional logic, but branching is expressed as nested function calls rather than explicit graph edges. The resulting code is harder to read, test, and extend.
- **Plain Python**: Custom orchestration with if/else. Works, but loses the benefits of LangGraph's state management, execution tracing, and graph visualization.

Why LangGraph: The HITL routing requirement makes this a branching workflow, not a linear pipeline. LangGraph provides: (1) explicit node and edge declarations that document the workflow topology, (2) typed state that enforces a contract between nodes, (3) conditional edges as a first-class concept, (4) built-in execution tracing for debugging. Adding a new node (e.g., a logging node, a feedback collection node) requires adding one node definition and one edge — no refactoring of existing code.

3.7 Confidence Threshold = 0.4

Decision: Set the HITL escalation threshold at 0.4 average retrieval confidence.

Alternatives considered:

- **0.2**: Very permissive — the system would only escalate when retrieval is nearly random. This risks returning hallucinated answers for moderately off-topic queries.
- **0.6**: Very conservative — would escalate a significant fraction of legitimate queries, overwhelming the human operator and defeating the purpose of automation.

Why 0.4: This threshold was calibrated by observing relevance scores on the sample FAQ corpus. In-domain queries (e.g., "return policy") consistently score 0.5-0.8 average confidence. Out-of-domain queries (e.g., "billing dispute for order #98765") score 0.2-0.4. A threshold of 0.4 cleanly separates these populations. In production, this threshold should be tuned per deployment using labeled query logs and precision/recall analysis on escalation decisions.

Trade-off: The threshold is corpus-dependent. A different document corpus with different vocabulary density would produce a different score distribution, potentially requiring recalibration.

---

4. LangGraph Workflow Explanation

4.1 Why Graphs Over Linear Chains

A linear chain (`retrieve → generate → respond`) works for the happy path but cannot express the conditional HITL routing without bolting on external control flow. When you add a condition like "if confidence < 0.4, call a different function and return its output instead", you are no longer describing a chain — you are describing a directed graph with branching edges.

LangGraph makes this explicit. The workflow topology is declared once in `build_graph()` and compiled into an executable. When the graph runs, LangGraph handles node sequencing, state passing, and conditional edge evaluation internally. The developer's intent (retrieve, then generate, then maybe escalate) is readable directly from the code without tracing through nested function calls.

4.2 Node Responsibilities

`retrieve_node` is the data-fetching layer. It loads the ChromaDB vector store, embeds the query, performs similarity search, and writes two outputs to state: the retrieved chunk texts (`retrieved_chunks`) and the average relevance score (`confidence`). It does not invoke the LLM and does not make routing decisions — it is purely a data retrieval step.

`generate_node` is the reasoning layer. It reads the query and retrieved chunks from state, constructs a structured prompt, invokes the LLM, and writes the response to state. The prompt engineering is critical: the system message instructs the LLM to answer only from provided context, to say "I don't know" when context is insufficient, and to include "ESCALATE" for topics requiring human judgment. This prompt design creates the feedback signal that the routing layer uses.

`hitl_node` is the human escalation layer. It only executes when the routing function determines that the LLM's response is unreliable. It presents the full context (query, LLM response, confidence) to a human operator, collects their response, and overwrites the LLM's response in state. After `hitl_node` , the graph always terminates — the human's response is the final answer.

4.3 State Transitions and Data Integrity

State integrity is maintained by LangGraph's merge semantics: each node returns only the fields it modifies, and these are shallow-merged into the current state. This guarantees:

- `retrieve_node` cannot accidentally overwrite `query` (it does not return it)
- `generate_node` cannot modify `confidence` (it does not return it)
- `hitl_node` explicitly overwrites `response` because the human's answer supersedes the LLM's

The `SupportState` TypedDict provides compile-time documentation of the state schema, making it clear which fields exist and what types they carry. While Python's TypedDict does not enforce types at runtime, it serves as a contract between node developers.

4.4 Conditional Routing as Intent Detection

The routing function `route_after_generate()` implements a form of implicit intent detection. It does not classify the query's intent directly — instead, it observes the system's behavior (retrieval confidence, LLM self-assessment) and infers whether the query is within the system's competence:

- High confidence + clean response → the system understood the query and found relevant information → route to END
- Low confidence → the query does not match the indexed knowledge base → route to HITL
- LLM signals uncertainty → the query matches the knowledge base but the content is insufficient → route to HITL

This is more robust than explicit intent classification because it does not require training a separate classifier or maintaining an intent taxonomy.

---

5. HITL Implementation

5.1 Role of Human Intervention in AI Systems

Every AI system has a boundary beyond which its outputs become unreliable. For a RAG system, this boundary is defined by the coverage of its knowledge base and the reasoning capability of its LLM. Queries that fall outside this boundary — novel topics, ambiguous questions, sensitive matters — produce responses that range from incomplete to outright harmful.

Human-in-the-Loop provides a structured mechanism for the system to recognize and acknowledge its limitations. Rather than presenting a hallucinated answer as truth, the system explicitly says "I cannot confidently answer this" and routes to a human who can. This preserves user trust and creates a feedback loop for system improvement.

5.2 Current Implementation

The current HITL implementation is a synchronous CLI interface:

1. **Detection**: `should_escalate()` evaluates three conditions (confidence threshold, "escalate" keyword, uncertainty phrases).
2. **Presentation**: The `hitl_node` displays the query, LLM's attempted response, and confidence score to the terminal.
3. **Collection**: `input()` blocks until the human types a response.
4. **Integration**: The human response replaces the LLM response in state and is returned to the user, prefixed with `[Human Agent]`.

5.3 Benefits

- **Accuracy**: Human operators provide authoritative answers for queries outside the knowledge base, maintaining response quality.
- **Trust**: Users see that the system acknowledges uncertainty rather than guessing, building confidence in the system's reliable responses.
- **Edge case handling**: Novel queries, policy exceptions, and complex scenarios are handled correctly without requiring system updates.
- **Data collection**: Escalated queries identify gaps in the knowledge base, providing a natural signal for which documents to add.

5.4 Limitations

- **Latency**: The blocking `input()` call introduces unbounded latency. A customer waiting for a response would experience a delay ranging from seconds (attentive operator) to hours (operator busy or unavailable).
- **Scalability**: A single terminal can serve one operator handling one query at a time. This does not scale to production query volumes.
- **Cost**: Every escalated query requires human time — at scale, this creates a significant labor cost. The escalation threshold directly controls this cost/quality trade-off.
- **Availability**: The system hangs if no human is available. There is no timeout, no fallback, and no queue management.

5.5 Production-Grade HITL Design

In production, the `hitl_node` would integrate with an async messaging system:

1. **Trigger**: `hitl_node` sends a structured message (query, context, confidence, session ID) to a message queue (e.g., AWS SQS, Redis Streams, or a Slack webhook).
2. **Interim response**: The graph immediately returns: "Your query has been escalated to a support specialist. You will receive a response within [SLA] hours."
3. **Agent UI**: Support agents see escalated queries in a dashboard, prioritized by wait time and topic.
4. **Resolution**: The agent crafts a response in the UI, which triggers a callback to the system.
5. **Delivery**: The system delivers the agent's response to the customer via their original channel (email, chat, SMS).

This architecture decouples escalation from resolution, enabling SLA management, workload balancing, and agent specialization.

---

6. Challenges & Trade-offs

6.1 Retrieval Accuracy vs. Speed

Using `all-MiniLM-L6-v2` provides fast embedding (~14ms/query on CPU) at the cost of some retrieval accuracy compared to larger models. For the FAQ corpus, where topics are semantically distinct (returns vs. shipping vs. billing), MiniLM's accuracy is sufficient. For a corpus with subtly different topics (e.g., distinguishing "return" from "exchange" from "replacement"), a larger model would be necessary.

6.2 Chunk Size vs. Context Quality

The 500-character chunk size creates a tension: smaller chunks improve retrieval precision (the search returns exactly the relevant paragraph) but reduce context completeness (the LLM may need information from an adjacent paragraph). The 50-character overlap partially mitigates this, but some information loss at chunk boundaries is inevitable. The alternative — larger chunks — would reduce precision and make the confidence score less discriminating.

6.3 Cost vs. Performance

GPT-3.5-turbo costs ~$0.002 per response for a typical 500-token prompt/response. This is negligible for demo purposes but scales linearly with query volume. At 10,000 queries/day, LLM costs alone 20/day. Switching to a local model (e.g., Mistral-7B via Ollama) eliminates API costs but requires GPU infrastructure and careful quality validation.

6.4 HITL Latency vs. Response Quality

Every HITL escalation introduces latency but guarantees a correct response. The confidence threshold (0.4) controls this trade-off: lowering it reduces escalation volume (faster responses on average) but increases the risk of hallucinated answers; raising it increases escalation volume

(more correct responses) but burdens the human operator and slows average response time.

6.5 Real Challenge: ChromaDB Score Normalization

During development, a significant challenge arose with ChromaDB's relevance score semantics. The `similarity_search_with_relevance_scores` method returns scores in [0, 1] where higher is more similar, but the underlying distance metric (L2 or cosine) varies by configuration, and the normalization behavior changed across ChromaDB versions. The solution was to explicitly L2-normalize all embeddings (`normalize_embeddings=True` in `HuggingFaceEmbeddings`), ensuring cosine similarity semantics regardless of ChromaDB's internal distance metric. This made confidence scores consistent and interpretable across different deployment environments.

---

7. Testing Strategy

7.1 Testing Approach

Testing follows a three-layer strategy:

1. **Unit testing**: Each module (`ingestion.py` , `vector_store.py` , `hitl.py`) is testable in isolation. Functions accept explicit inputs and return explicit outputs with no hidden state.
2. **Integration testing**: The `run_query()` function tests the complete pipeline end-to-end, from query to response.
3. **Behavioral testing**: Demo queries are designed to exercise both the normal path (confident response) and the escalation path (HITL).

7.2 Sample Test Queries

#	Query	Expected Route	Expected Behavior
1	"What is your return policy?"	END (normal)	Returns return policy details from the FAQ; confidence > 0.5
2	"How do I reset my password?"	END (normal)	Returns password reset steps; confidence > 0.5
3	"What are your business hours?"	END (normal)	Returns support hours; confidence > 0.5
4	"How much does express shipping cost?"	END (normal)	Returns $9.99 express shipping info; confidence > 0.5
5	"Do you offer a warranty?"	END (normal)	Returns warranty information; confidence > 0.5
6	"How do I cancel my subscription?"	END (normal)	Returns subscription cancellation steps; confidence > 0.5
7	"What is the meaning of life?"	HITL (edge)	Completely off-topic; confidence < 0.4; triggers escalation
8	"Tell me about quantum computing"	HITL (edge)	Off-domain query; no relevant chunks; triggers escalation
9	"I need help with a billing dispute for order #98765"	HITL (trigger)	System prompt instructs LLM to escalate billing disputes
10	"My account has been hacked, what do I do?"	HITL (trigger)	System prompt instructs LLM to escalate security issues

7.3 Validation Criteria

For each test query:

- **Normal path**: Verify `escalate == False` , `confidence > 0.4` , and response contains relevant information from the FAQ.
- **HITL path**: Verify `escalate == True` , and the system correctly prompts for human input.
- **All paths**: Verify no exceptions are thrown, state schema is complete, and print output provides clear execution trace.

---

8. Future Enhancements

8.1 Multi-Document Support with Metadata Filtering

The current system treats all chunks as belonging to a single flat collection. In production, documents would be tagged with metadata (product line, department, effective date), and retrieval would be scoped using metadata filters. ChromaDB supports this natively via its `where` parameter: `vector_store.similarity_search(query, k=3, filter={"department": "billing"})` . This prevents cross-contamination between unrelated document domains.

8.2 Feedback Loop for Retrieval Improvement

Human responses from HITL escalations represent ground-truth answers for queries the system failed to handle. These can be fed back into the system in two ways: (1) adding the human's response as a new document chunk, expanding the knowledge base coverage; (2) using the (query, correct_answer) pair as a training signal for embedding model fine-tuning, improving retrieval for similar future queries.

8.3 Conversation Memory with LangGraph Persistence

The current system handles single-turn queries. LangGraph supports state persistence via checkpointing, which enables multi-turn conversations where the system remembers previous queries and responses within a session. This would allow follow-up questions like "Can you elaborate on the return timeline?" without repeating context.

8.4 Streamlit / FastAPI Deployment

The CLI interface serves its purpose for demos and development, but production deployment requires a web interface. Two paths:

- **Streamlit**: Rapid prototyping of a chat UI with minimal code. Suitable for internal tools and demos.
- **FastAPI**: REST API for integration into existing customer-facing applications (websites, mobile apps, CRM systems).

8.5 Evaluation Metrics: RAGAS Framework

Systematic evaluation requires metrics beyond manual inspection. The RAGAS (Retrieval Augmented Generation Assessment) framework provides four automated metrics: (1) **Faithfulness** — does the response contain only information from the retrieved context? (2) **Answer Relevancy** — is the response relevant to the query? (3) **Context Precision** — are the retrieved chunks relevant? (4) **Context Recall** — do the retrieved chunks contain the information needed to answer? These metrics can be computed over a labeled test set to quantitatively track system quality as the knowledge base and configuration evolve.

Document version 1.0 — April 2026